Week 7 — Text Data & Modules

Code the Dream · Python Intro

Session Overview

Explore — ~30 min

- Reading and writing files with `open()` and `with`
- Parsing CSVs with `csv.DictReader`
- `os` and `datetime` from the standard library
- When to import vs. write your own

Apply — ~30 min

- Code-along: read a text file line by line
- Code-along: parse a CSV into a list of dicts
- Code-along: expense report generator

Programs That Work With Real Data

Every program you've written so far used data you typed into the code — or data the user typed at runtime.

This week your programs learn to **read data from files** and **write results back to disk**.

Reading a File With `with`

The `with` statement opens a file and closes it automatically.

```
with open("notes.txt", "r") as f:
    for line in f:
        print(line.strip())
```

File Modes

Mode	Meaning
`"r"`	Read (default) — file must exist
`"w"`	Write — creates or overwrites
`"a"`	Append — adds to end of existing file

```
with open("output.txt", "w") as f:
    f.write("Hello from Python!\n")
```

Check for Understanding #1

You want to add a new entry to an existing log file without erasing previous entries. Which mode should you use?

A) `"r"` — read
B) `"w"` — write
C) `"a"` — append
D) `"rw"` — read and write

Parsing CSVs With `csv.DictReader`

`DictReader` reads each row as a dictionary — keys come from the header row.

```
import csv

with open("students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["name"], row["score"])
```

Connecting CSV to Data Structures

Load a CSV into a list of dicts — then loop over it with your Week 4 skills.

```
import csv

students = []
with open("students.csv", "r") as f:
    for row in csv.DictReader(f):
        students.append({
            "name": row["name"],
            "score": float(row["score"])
        })
```

The `os` Module

`os` lets you work with files and directories from your script.

```
import os

print(os.getcwd())                          # current directory
print(os.path.exists("data/file.csv"))      # True or False
path = os.path.join("..", "data", "file.csv")
print(path)                                 # ../data/file.csv
```

Check for Understanding #2

Why use `os.path.join("..", "data", "file.csv")` instead of just writing `"../data/file.csv"` directly?

A) `os.path.join` is faster
B) The string `"../data/file.csv"` only works on Mac and Linux; `os.path.join` builds the correct path for any OS
C) Python cannot read strings with `/` in them
D) There is no difference — both always work

The `datetime` Module

Work with today's date and format it for output.

```
from datetime import datetime

today = datetime.now()
print(today.strftime("%B %d, %Y"))   # June 03, 2026
```

Check for Understanding #3

When should you reach for a standard library module instead of writing your own solution?

A) Always — writing your own code is never worth it
B) When the problem is common and already well-solved, like file paths, dates, or CSV parsing
C) Only when the task involves the internet
D) Only when the assignment requires it

Time to Apply

Let's put file I/O and modules to work.

Mentor 2 takes over

Assignment 7 Overview

Part 1 — Four warmup scripts (data files in `week-7/data/`)

- `warmup1.py` — read `notes.txt` line by line with line numbers
- `warmup2.py` — parse `students.csv` with `DictReader`
- `warmup3.py` — use `os`: getcwd, path.exists, path.join
- `warmup4.py` — print today's date with `datetime.strftime`

Part 2 — Expense Report Generator

- Read `expenses.csv`, filter to "Food" category
- Write a formatted report to `food_report.txt`
- Use `os.path.exists()` and `datetime.now()`

Code-Along 1: Read a File Line by Line

Print each line with its line number.

```
with open("../data/notes.txt", "r") as f:
    for i, line in enumerate(f, start=1):
        print(f"Line {i}: {line.strip()}")
```

Code-Along 2: Parse a CSV

Read each row as a dictionary.

```
import csv

with open("../data/students.csv", "r") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(f"{row['name']}: {row['score']}")
```

Code-Along 3: Expense Report Starter

Check the file exists, then read and filter.

```
import csv, os
from datetime import datetime

path = os.path.join("..", "data", "expenses.csv")
if not os.path.exists(path):
    print("Error: expenses.csv not found.")
else:
    with open(path, "r") as f:
        rows = [r for r in csv.DictReader(f)
                if r["category"] == "Food"]
    print(f"Found {len(rows)} food expenses.")
```

Extension Challenge

Ungraded extension: Modify your expense report generator to work for **any category**, not just `"Food"`.

Ask the user which category they want, then generate a report named `{category.lower()}_report.txt`.

Running it for `"Transport"` should produce a well-formatted `transport_report.txt`.

Wrap-Up

Today you learned:

- `open()` and `with` for safe file reading and writing
- File modes: `"r"`, `"w"`, `"a"`
- `csv.DictReader` for parsing CSVs into lists of dicts
- `os` and `datetime` from the standard library

This week: Complete warmups 1–4 and the Expense Report Generator. Include `food_report.txt` in your PR.

Next week: Errors and debugging — writing code that handles the unexpected.